



MongoDB 사용 및 컬렉션 설계 배경 정리

- 1. MongoDB를 선택한 이유
- 2. 컬렉션 구조 변경 및 통합 이유
- 3. 인덱싱 전략

1. MongoDB를 선택한 이유

- 비정형 데이터(Non-relational Data) 구조에 적합

이미지 경로, 음악 생성 결과, 작업 상태 정보 등은 사용자마다 구조와 개수, 형태가 달라지는 유동적인 데이터입니다.

MongoDB는 JSON 기반 문서 구조를 통해 리스트나 중첩 객체 등 비정형 데이터를 자연스럽게 유연하게 표현할 수 있어, 관계형 데이터베이스보다 적합합니다.

- 문서 단위 저장 방식이 도메인 모델과 잘 맞음

사용자 요청은 풍등(lantern) 단위로 생성 및 조회되며, 하나의 풍등에 이미지, 음악, 태스크 상태 등의 정보가 함께 묶여 있습니다.

MongoDB는 연관된 데이터를 하나의 문서에 통합해 저장할 수 있어, 도메인 구조와 직관적으로 잘 맞고 데이터 일관성 관리도 용이합니다.

- 조회 중심 서비스 구조에 최적화됨

본 서비스는 풍등을 생성하는 것보다 조회 요청이 훨씬 많은 구조입니다.

MongoDB는 단건 문서 조회 속도가 빠르고, 인덱싱을 통해 빠른 필터링이 가능해 조회가 많은 서비스에 적합한 성능 특성을 가집니다.

2. 컬렉션 구조 변경 및 통합 이유

2-1. 컬렉션 구조 변경 이유

초기 설계 (기능별 분리)

- 사용자 정보 + 이미지 정보 (images)
- 음악 정보 (musics)
- 파노라마 정보 (panoramas) ← 삭제됨(기능이 삭제됨)

구조 변경 배경

- 파노라마 기능 제거
- 풍등 단위로 생성/조회가 이루어져서 단일 컬렉션 구조가 더 적합



단일 컬렉션 구조가 더 적합한 이유

- 풍등 단위로 데이터가 논리적으로 묶여 있음
한 개의 풍등(lantern)은 이미지, 음악, 생성 태스크 등 하나의 의미 있는 단위를 구성함
→ 개별 정보를 다른 컬렉션에 흩어놓기보다는 하나의 문서로 묶는 것이 도메인 구조에 직관적
- API 응답 속도 향상
주요 API가 풍등 단위로 호출되며(GET /lanterns/{id}), 이미지·음악 정보를 함께 반환해야 함
→ 단일 컬렉션이면 한 번의 조회로 모든 정보 획득 가능
→ 다중 컬렉션이면 join 또는 다중 쿼리 필요 → 복잡도 증가 + 응답 지연 발생
- 데이터 일관성 유지 용이
하나의 풍등 생성 시 이미지와 음악 태스크 상태도 함께 저장되므로, 단일 문서로 관리하면 트랜잭션 없이도 데이터 간 정합성 확보 가능
- 변경/삭제/조회 로직 간소화
풍등을 삭제하거나 변경할 때 연관된 이미지/음악/태스크를 한 번에 처리 가능
→ 다중 컬렉션이면 별도 참조 키 관리 및 연쇄 삭제 구현 필요

- 이미지 및 음악 정보 모두 string 위주의 경량 데이터 → 하나로 합쳐도 성능 저하 없음

2-2. 최종 구조: lanterns 컬렉션

```
Lantern {
  _id: ObjectId,
  lantern_id: string,
  user_name: string,
  is_public: bool,
  created_at: datetime,
  images: [ImageInfo],
  musics: [MusicInfo],
  music_tasks: [string],
  music_statuses: [MusicStatusInfo]
}
```

```
ImageInfo {
  s3_path: string,
  filename: string,
  extension: string,
  size: number
}
```

```

}

MusicInfo {
  description: string,
  s3_path: string,
  created_at: datetime
}

MusicStatusInfo {
  image_s3: string,
  task_id: string,
  status: string,      // e.g. pending, processing, completed, failed
  s3_key?: string      // optional, 생성 성공 시에만 존재
}

```

3. 인덱싱 전략

3-1. 인덱싱 적용

- 대부분의 API가 lantern_id 기준으로 단건 조회
 - /api/v1/lanterns/{lantern_id}
 - /api/v1/lanterns/{lantern_id}/music
 - SSE 상태 확인용 /status API 등
- 조회 성능 향상 및 안정성 확보를 위해 lantern_id에 단일 인덱스 적용

```

# pymongo 기준
db.lanterns.create_index("lantern_id", unique=True)

```

3-2. 인덱싱 효과

- MongoDB에서 lantern_id 필드에 단일 인덱스를 적용함
- 총 100,000건의 더미 데이터를 삽입하여 실험 환경 구축
- 그 중 100개의 랜덤한 풍등 ID를 대상으로 조회 성능 실험 수행
- 실험 지표는 다음과 같음
 - 응답 속도 (평균/최대)
 - 문서 스캔 수
 - 쿼리 실행 단계 (COLLSCAN vs IXSCAN 등)
- 실험 대상 API는 실제 운영 환경에서 가장 자주 호출되는
 - 단건 조회 API: /api/v1/lanterns/{lantern_id}
 - 이 API를 기준으로 삼은 이유

- 전체 서비스의 조회 요청 대부분이 내부적으로 findByLanternId 로직을 공유하고 있음
- 따라서 해당 API 기준의 측정 결과가 전체 서비스 성능을 대표함

항목	인덱스 미적용 (1차)	인덱스 적용 후 (2차)
평균 응답 시간	1.17ms	1.38ms
최대 응답 시간	23.33ms	1.94ms
인덱스 단계	EXPRESS_IXSCAN	EXPRESS_IXSCAN
문서 스캔 수	1.00개	1.00개

쿼리 실행 성능 측정 중...

[100건 테스트 결과]

- 평균 응답 시간: 1.17ms
- 최소 응답 시간: 0.67ms
- 최대 응답 시간: 23.33ms
- 사용된 인덱스 단계: EXPRESS_IXSCAN
- 평균 문서 스캔 수: 1.00개

인덱스 생성 완료

쿼리 실행 성능 측정 중...

[100건 테스트 결과]

- 평균 응답 시간: 1.38ms
- 최소 응답 시간: 1.15ms
- 최대 응답 시간: 1.94ms
- 사용된 인덱스 단계: EXPRESS_IXSCAN
- 평균 문서 스캔 수: 1.00개



해석 및 결론

- 평균 응답 시간은 인덱스 유무에 따라 큰 차이는 없었으나, 최대 응답 시간에서는 약 10배 이상의 차이가 발생하였다.
- 이는 MongoDB 내부의 메모리 캐시로 인해 평균 속도는 유사하게 유지되었지만, 인덱스가 없을 경우 디스크 I/O 상황이나 랜덤한 위치 탐색 시 성능 일관성이 떨어진다는 점을 보여준다.
- 특히 본 프로젝트는 풍등 단위로 자주 조회되며, SSE, Polling 등 실시간성 요구가 많은 서비스 특성상, 최악의 응답 시간(latency)을 줄이는 것이 사용자 경험에 중요하다.



결론

따라서 단일 인덱스 적용은 평균 속도뿐만 아니라, 전체 성능 안정성과 최악 케이스 방어를 위한 필수 조건이라고 판단했습니다.